Modellierung mit Genetischen Algorithmen

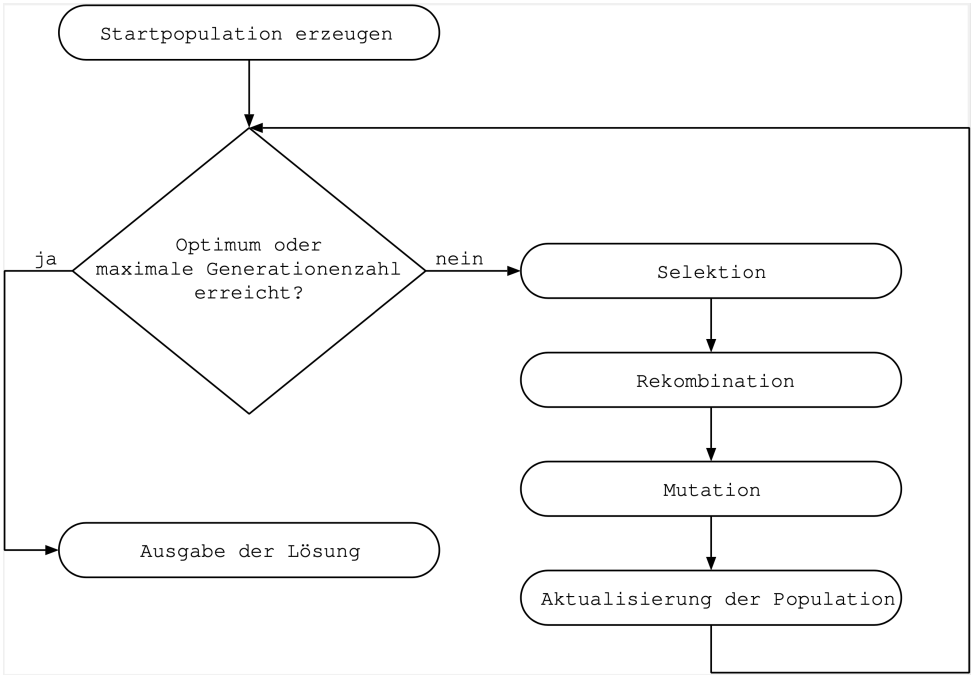
Carsten Gips (HSBI)

TL;DR
Lokale Suchverfahren: Nur das Ergebnis zählt!
Evolutionäre Algorithmen sind lokale Suchverfahren, wobei gleichzeitig an mehreren Stellen im Problemraum gesucht wird. Sie bedienen sich Mechanismen aus der Evolution: Es gibt eine Population von Individuen, die jedes das Problem kodieren ("vollständige Zustandsbeschreibung") und damit im Laufe der Suche zu einer möglichen Lösung werden können.
Die Individuen werden mit Hilfe einer Fitnessfunktion bewertet, wie gut sie bereits an das Problem angepasst sind (bzw. wie sehr sie bereits der gesuchten Lösung entsprechen). Über eine fitnessproportionale Selektion werden Individuen ausgewählt, aus denen mittels Rekombination (auch "Crossover" genannt) neue Individuen mit Eigenschaften der Eltern erzeugt werden. Über eine Mutation werden dann noch Elemente der neuen Individuen leicht verändert, bevor diese zur neuen Population werden …
Durch das Anwenden von Rekombination und Mutation springt man im Problemraum umher. Auch wenn als Basis die fitteren (angepassteren) Individuen dienen, kann es wie bei allen lokalen Suchverfahren vorkommen, dass sich der Algorithmus in lokalen Minima (bzw. lokalen Maxima, je nach Richtung der Optimierung) festfrisst.
Videos

- VL Modellierung mit EA/GA

---

1 EA – Allgemeiner Ablauf



2 Kodierung Individuen

- Binäre Lösungsrepräsentation (Bitstring): $\mathbf{g} = (g_1, \ldots, g_m) \in \{0,1\}^m$
  - String gliedert sich in n Elemente (mit $n \leq m$)
    => jedes Segment entspricht einer Problemvariablen
  - Dekodierungsfunktion $\Gamma : \{0,1\}^m \to \mathbb{R}^n$

  Alle relevanten Aspekte des Problems müssen in die Codierung einfließen!
  Bei ES hat man einen Vektor mit reellen Zahlen, wobei jeder Eintrag einen Parameter des Problems darstellt.
  Eine Dekodierungsfunktion benötigt man entsprechend nicht.
  Bei der Erzeugung der Startpopulation werden die Individuen **zufällig** (mit zufälligen Werten) initialisiert.

- Fitnessfunktion Φ ordnet jedem Individuum $\mathbf{g}_i$ eine reelle Zahl zu:

$$\Phi(\mathbf{g}_i) = F(\Gamma(\mathbf{g}_i)) - w \cdot \sum_j (Z_j(\Gamma(\mathbf{g}_i)))^2$$

  - Zielfunktion F: wie sehr genügt ein Individuum bereits dem Optimierungproblem
  - Strafterme Z_j: Anreicherung der Optimierung mit weiteren Informationen
  - Gewichte w: statisch oder dynamisch (Abkühlen)

  Die Wahl einer guten Fitnessfunktion ist oft eine Herausforderung, aber dennoch wichtig, da damit die Suche gesteuert wird!

3 Selektion: Erstelle Matingpool mit μ Individuen

- Fitnessproportionale Selektion (*Roulette Wheel Selection*):
  Auswahlwahrscheinlichkeit für Individuum $\mathbf{g}_k$:

$$p_{sel}(\mathbf{g}_k) = \frac{\Phi(\mathbf{g}_k)}{\sum_j \Phi(\mathbf{g}_j)}$$

  => Voraussetzung: positive Fitnesswerte

- Turnier-Selektion (*Tournament Selection*):
  - Turniergröße ξ
  - Turnier: ziehe ξ Individuen gleichverteilt (mit Zurücklegen!) und kopiere fittestes Individuum in den Matingpool
  - Führe μ Turniere durch

Hinweis: Es gibt noch viele weitere Selektionsmechanismen. Die vorgestellten sind in der Praxis am gebräuchlichsten.

Über die Selektion wird der sogenannte “Selektionsdruck” aufgebaut: Wie gut muss ein Individuum sein (im Vergleich zu den restlichen Individuen in der Population), damit es eine Chance zur Reproduktion erhält? Dürfen sich nur die “Guten” fortpflanzen, oder erhalten auch die “Schlechten” eine gewisse Chance?

Da jedes Individuum einen Punkt im Suchraum darstellt, beeinflusst die Wahl der Selektion die Geschwindigkeit der Suche, begünstigt u.U. aber auch ein eventuelles Festfahren in lokalen Minima. Dies kann beispielsweise geschehen, wenn immer nur die “Guten” selektiert werden, aber die “Guten” der Population sich in der Nähe eines lokalen Minimums befinden. Dann werden auch die Nachfolger sich wieder dort aufhalten.

4 Crossover: Erzeuge zwei Nachkommen aus zwei Eltern

Festlegung der Crossover-Wahrscheinlichkeit p_{cross} (typisch: $p_{cross} \geq 0.6$)

1. Selektiere Eltern $\mathbf{g}_a$ und $\mathbf{g}_b$ **gleichverteilt** aus Matingpool

2. Zufallsexperiment:

   - mit $1 - p_{cross}$: Kinder identisch zu Eltern (kein Crossover)
   - mit p_{cross}: Crossover mit $\mathbf{g}_a$ und $\mathbf{g}_b$
     - Ziehe i gleichverteilt mit $1 < i < m$
     - Kinder aus $\mathbf{g}_a$ und $\mathbf{g}_b$ zusammenbauen:

       $$\mathbf{g}_c = (g_{a,1}, \ldots, g_{a,i},\ g_{b,i+1}, \ldots, g_{b,m})$$

       und

       $$\mathbf{g}_d = (g_{b,1}, \ldots, g_{b,i},\ g_{a,i+1}, \ldots, g_{a,m})$$

   => Trenne Eltern an gleicher Stelle auf, vertausche Bestandteile

3. Gehe zu Schritt 1, bis insg. μ Nachkommen

Anmerkung: Die Eltern werden jeweils in die Ausgangsmenge zurückgelegt.

Mit einer kleinen Wahrscheinlichkeit sind die Kinder also identisch zu den Eltern. Dies ist im Sinne der lokalen Suche wichtig, um bereits erreichte gute Positionen im Suchraum nicht zu verlieren: Es könnte sein, dass die Nachfolger alle schlechter sind …

Varianten: N-Punkt-Crossover, Shuffle-Crossover

Bei ES wird parameterweise gekreuzt. Dabei gibt es verschiedene Möglichkeiten: Übernahme eines Parameters von einem Elternteil, Verrechnen (beispielsweise Mitteln) der Werte beider Eltern, … Bei ES heißt “Crossover” deshalb oft “Rekombination”.

5 Mutation

- Mutationswahrscheinlichkeit p_{mut}
  (typische Werte: $p_{mut} = 0.01$ oder $p_{mut} = 0.001$)

- Für alle Individuen:
  - Mutiere jedes Gen eines Individuums mit p_{mut}:

$$g_i^{(t+1)} = \begin{cases} \neg g_i^{(t)} & \text{falls } \chi_i \leq p_{mut} \\ g_i^{(t)} & \text{sonst} \end{cases}$$

=>χ_i gleichverteilte Zufallsvariable (Intervall $[0, 1]$), für jedes Bit g_i neu bestimmen

Anmerkung: Die optimale Mutationsrate p^*_{mut} ist von Länge m des Bitstrings abhängig; annäherbar durch $p^*_{mut} \approx 1/m$.

Die beim Crossover erstellten Nachfolger liegen im Suchraum in der Nähe der Eltern. Durch die Mutationsrate bestimmt man, ob und wie weit sich ein Kind entfernen kann. Dies entspricht dem Bild des "Schüttelns" der Zustandslandschaft.

Bei ES unterscheidet man Mutationswahrscheinlichkeit und Mutationsrate. Es wird parameterweise mutiert.

6 Bewertungskriterien

Vorsicht: Es handelt sich um Zufallsexperimente. Wenn man nicht nur direkt nach einer Lösung sucht, sondern beispielsweise Parametereinstellungen oder die Wahl der Fitnessfunktion für ein Problem vergleichen will, muss man jeweils mehrere Experimente mit der selben Einstellung machen und Kenngrößen berechnen.

Geschwindigkeit: AES *Average Evaluations to a Solution*

$$\text{AES} = \frac{\sum_{i \in \text{erfolgreiche Läufe}} \text{Generationen von Lauf } i}{\text{Anzahl der erfolgreichen Läufe}}$$

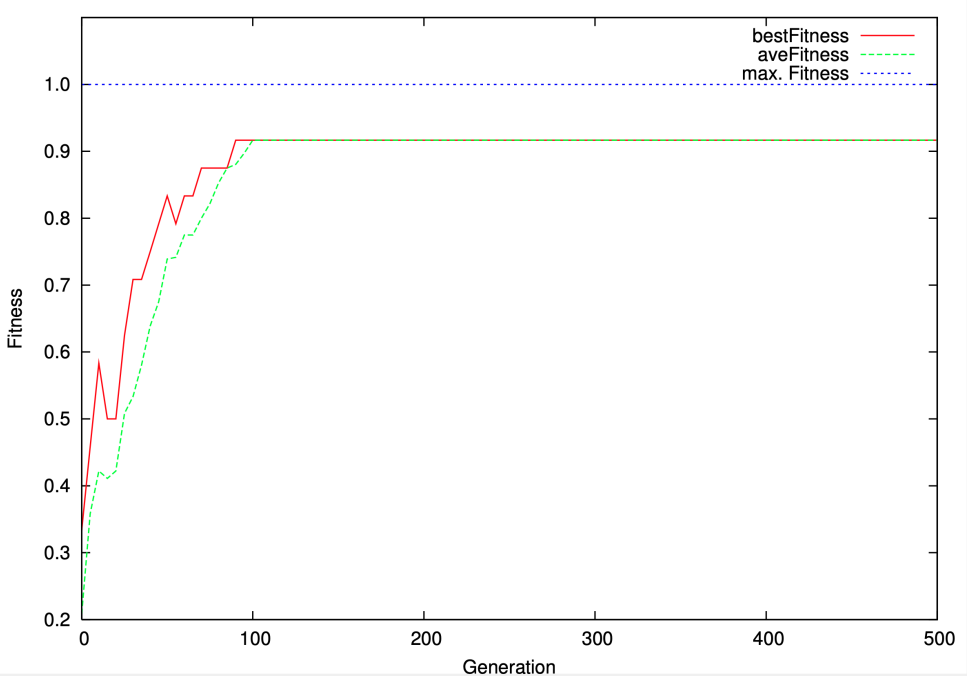
Die AES liegt im Intervall $[0, maxGen]$.

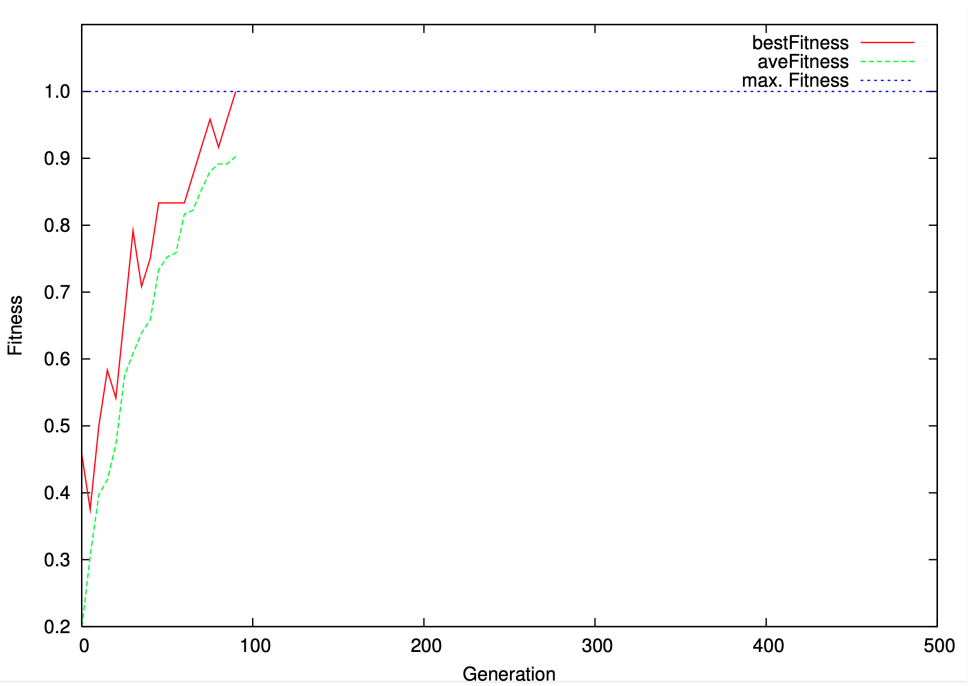
Lösungswahrscheinlichkeit: SR *Success Rate*

$$\text{SR} = \frac{\text{Anzahl der erfolgreichen Läufe}}{\text{Anzahl aller Läufe}}$$

Die SR liegt im Intervall $[0, 1]$.

7 Typische Läufe





- Populationsgröße $\mu = 15$
- Anzahl Nachfahren $\lambda = 100$
- Abbruch nach $maxGen = 200$ Generationen

Stochastischer Algorithmus! Ausreichend Wiederholungen durchführen und mitteln!

Hinweis: Die Parameter müssen problemabhängig gewählt werden. Zu hohe Werte für μ und λ führen dazu, dass man bei kleinen Problemen mit hoher Wahrscheinlichkeit bereits am Anfang eine Lösung "würfelt", also gar kein GA nutzt. Wenn dies allerdings nicht passiert, sorgt eine hohe Populationsgröße dafür, dass jeder Schritt sehr lange dauert. Die Abbruchgrenze ist ebenfalls mit Augenmaß zu wählen: Ein zu kleiner Wert sorgt für zu frühen Abbruch (keine Lösung!), ein zu hoher Wert sorgt beim Festfressen des Algorithmus für eine unnötige weitere "Suche" …

8 Wrap-Up

Lokale Suchverfahren: Nur das Ergebnis zählt!

- Evolutionäre Algorithmen:
  - Begriffe: Individuum, Population, Kodierung
  - Operationen: Selektion, Rekombination, Mutation
  - Bewertung mit Fitnessfunktion

9 Zum Nachlesen

- Russell und Norvig (2020): GA: Abschnitt 4.1.4
- Schwefel (1995): Originalarbeit zu Evolutionsstrategien
- Michalewicz (1996)
- Bäck (1996)
- Nissen (1997)

Lernziele

- k3: GA: Kodierung, Fitnessfunktion, Ablauf, Operatoren, Auswertung

Quizzes

- Selbsttest EA/GA (ILIAS)

Challenges

Sudoku

Ein 9×9-*Sudoku*-Rätsel soll mit einem GA gelöst werden.

Geben Sie für dieses Problem jeweils eine geeignete **Kodierung** der Individuen, passende Operatoren (**Crossover**, **Mutation**) und eine geeignete **Fitnessfunktion** an, damit das Problem mit einem GA gelöst werden kann. Begründen Sie Ihre Wahl!

Was würden Sie noch benötigen, um das Probleme mit Simulated Annealing lösen zu können?

Travelling Salesman Problem

Das *Travelling Salesman Problem* für 10 Städte, d.h. das Finden der kürzesten Route zwischen 10 Städten, soll mit einem GA gelöst werden.

Geben Sie für dieses Problem jeweils eine geeignete **Kodierung** der Individuen, passende Operatoren (**Crossover**, **Mutation**) und eine geeignete **Fitnessfunktion** an, damit das Problem mit einem GA gelöst werden kann. Begründen Sie Ihre Wahl!

Was würden Sie noch benötigen, um das Probleme mit Simulated Annealing lösen zu können?

Quellen

Bäck, T. 1996. *Evolutionary Algorithms in Theory and Praxis*. Oxford University Press.

Michalewicz, Z. 1996. *Genetic Algorithms + Data Structures = Evolution Programs*. Springer.

Nissen, V. 1997. *Einführung in Evolutionäre Algorithmen*. Vieweg+Teubner Verlag.

Russell, S., und P. Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson. http://aima.cs.berkeley.edu.

Schwefel, H. P. 1995. *Evolution and Optimum Seeking*. Wiley.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 5f0b0a3 (Remove Hugo: tooling: shift headings (use h1 as top-level headings), 2025-07-17)